```
!pip -q install opencv-python-headless scikit-learn

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.cluster import KMeans

print("Libraries imported successfully!")
print("OpenCV version:", cv2.__version__)
```

```
Libraries imported successfully!
OpenCV version: 5.0.0
```

```
uploaded = files.upload()

if not uploaded:
    raise ValueError("Please upload an image.")

filename = next(iter(uploaded))
file_bytes = np.frombuffer(uploaded[filename], np.uint8)
img = cv2.imdecode(file_bytes, cv2.IMREAD_COLOR)

if img is None:
    raise ValueError("Invalid image. Please upload JPG or PNG.")

# Image ko manageable size mein resize karna
h, w = img.shape[:2]
scale = min(1.0, 800 / max(h, w))

if scale < 1.0:
    img = cv2.resize(
        img, None, fx=scale, fy=scale,
        interpolation=cv2.INTER_AREA
    )

# OpenCV BGR image ko RGB mein convert karna
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Grayscale and Gaussian blur
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

print("Image loaded successfully:", filename)
print("Image dimensions:", img.shape)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

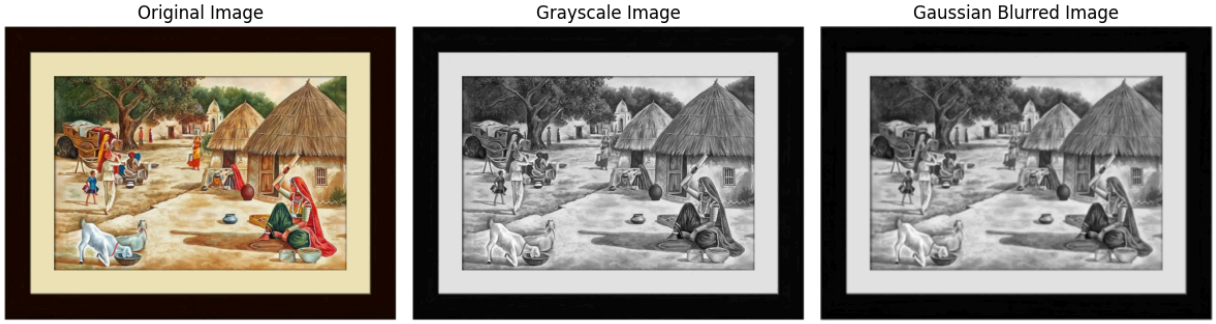
plt.subplot(1, 3, 3)
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Choose files village.jpg

village.jpg(image/jpeg) - 83279 bytes, last modified: 13/10/2026 - 100% done

```
Saving village.jpg to village.jpg
Image loaded successfully: village.jpg
Image dimensions: (597, 800, 3)
```




```
threshold_value = 127

_, global_thresh = cv2.threshold(
    blurred, threshold_value, 255, cv2.THRESH_BINARY
)

print("Global Thresholding completed.")
print("Threshold value:", threshold_value)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(blurred, cmap="gray")
plt.title("Blurred Grayscale")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Global Thresholding completed.
Threshold value: 127
```




```
otsu_value, otsu_thresh = cv2.threshold(
    blurred, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Otsu Thresholding completed.")
print("Automatically selected threshold:", otsu_value)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(blurred, cmap="gray")
plt.title("Input Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(otsu_thresh, cmap="gray")
plt.title(f"Otsu Threshold = {otsu_value:.0f}")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Otsu Thresholding completed.
Automatically selected threshold: 118.0
```



```
adaptive_thresh = cv2.adaptiveThreshold(
    blurred,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

print("Adaptive Thresholding completed.")

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(blurred, cmap="gray")
plt.title("Input Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Adaptive Thresholding completed.
```




```
# Binary image for foreground/background estimation
_, binary = cv2.threshold(
    blurred, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Remove small noise
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(
    binary, cv2.MORPH_OPEN, kernel, iterations=2
)

# Estimate sure background
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Estimate sure foreground
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(
```

```
    dist_transform,
    0.4 * dist_transform.max(),
    255,
    cv2.THRESH_BINARY
)

sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

# Create markers
num_labels, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

# Apply watershed on a copy of the color image
watershed_markers = cv2.watershed(img.copy(), markers)

# Display boundaries in red
watershed_result = img_rgb.copy()
watershed_result[watershed_markers == -1] = [255, 0, 0]

print("Watershed Segmentation completed.")
print("Detected marker regions:", num_labels - 1)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(watershed_result)
plt.title("Watershed Boundaries in Red")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Watershed Segmentation completed.
Detected marker regions: 7
```




```
# Convert image pixels into a 2D array
pixels = img_rgb.reshape((-1, 3)).astype(np.float32)

K = 3

kmeans = KMeans(
```

```
    n_clusters=K,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixels)
centers = np.uint8(kmeans.cluster_centers_)

segmented = centers[labels].reshape(img_rgb.shape)

print("K-Means Clustering completed.")
print("Number of clusters:", K)
print("Cluster centers (RGB):")
print(centers)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(segmented)
plt.title(f"K-Means Segmentation (K={K})")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
K-Means Clustering completed.
Number of clusters: 3
Cluster centers (RGB):
[[229 217 177]
 [ 33  18   8]
 [149 113  76]]
```




```
plt.figure(figsize=(15, 10))

results = [
    ("Original Image", img_rgb),
    ("Global Thresholding", global_thresh),
    ("Otsu's Thresholding", otsu_thresh),
    ("Adaptive Thresholding", adaptive_thresh),
    ("Watershed Segmentation", watershed_result),
    ("K-Means Clustering", segmented)
]
```

```
for i, (title, result) in enumerate(results, start=1):
    plt.subplot(2, 3, i)

    if result.ndim == 2:
        plt.imshow(result, cmap="gray")
    else:
        plt.imshow(result)

    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()

print("Comparative analysis completed.")
```




```
Comparative analysis completed.
```

```
print("=" * 70)
print("COMPARISON OF IMAGE SEGMENTATION TECHNIQUES")
print("=" * 70)

comparison = [
    ("Global Thresholding",
     "Uses one fixed threshold",
     "Simple images with clear contrast"),
    ("Otsu's Thresholding",
     "Automatically selects a global threshold",
     "Images with distinguishable intensity groups"),
    ("Adaptive Thresholding",
     "Uses local thresholds",
     "Uneven illumination"),
    ("Watershed",
     "Uses markers and image boundaries",
     "Separating touching objects"),
    ("K-Means Clustering",
     "Groups pixels by color similarity",
     "Color-based region segmentation")
]
```

```
for method, principle, application in comparison:
    print("\nMethod:", method)
    print("Principle:", principle)
    print("Suitable for:", application)

print("\nObservation:")
print(
    "The visual quality of each method depends on the image, "
    "lighting, contrast, and object-background differences."
)
```

```
======================================================================
COMPARISON OF IMAGE SEGMENTATION TECHNIQUES
======================================================================

Method: Global Thresholding
Principle: Uses one fixed threshold
Suitable for: Simple images with clear contrast

Method: Otsu's Thresholding
Principle: Automatically selects a global threshold
Suitable for: Images with distinguishable intensity groups

Method: Adaptive Thresholding
Principle: Uses local thresholds
Suitable for: Uneven illumination

Method: Watershed
Principle: Uses markers and image boundaries
Suitable for: Separating touching objects

Method: K-Means Clustering
Principle: Groups pixels by color similarity
Suitable for: Color-based region segmentation

Observation:
The visual quality of each method depends on the image, lighting, contrast, and object-backgro
```

```
cv2.imwrite("global_threshold.png", global_thresh)
cv2.imwrite("otsu_threshold.png", otsu_thresh)
cv2.imwrite("adaptive_threshold.png", adaptive_thresh)
cv2.imwrite(
    "watershed_segmentation.png",
    cv2.cvtColor(watershed_result, cv2.COLOR_RGB2BGR)
)
cv2.imwrite(
    "kmeans_segmentation.png",
    cv2.cvtColor(segmented, cv2.COLOR_RGB2BGR)
)

print("All output images saved successfully.")

files.download("global_threshold.png")
files.download("otsu_threshold.png")
files.download("adaptive_threshold.png")
files.download("watershed_segmentation.png")
files.download("kmeans_segmentation.png")
```

```
All output images saved successfully.
```

```
qa = [
    (
        "Q1. What is image segmentation? Why is it fundamental in computer vision?",
```

```
            "Image segmentation is the process of dividing an image into meaningful "
            "regions or objects based on properties such as intensity, color, texture "
            "or boundaries. It is fundamental because it isolates regions of interest "
            "and supports object detection, recognition and image analysis."
        ),
        (
            "Q2. Differentiate between image segmentation and image classification.",
            "Image classification assigns a class label to an image or its content. "
            "Image segmentation assigns labels to individual pixels or regions, "
            "showing where different objects or areas are located."
        ),
        (
            "Q3. Explain Global, Otsu's and Adaptive Thresholding.",
            "Global Thresholding uses one fixed threshold for the whole image. "
            "Otsu's Thresholding automatically selects a threshold based on the "
            "image intensity histogram. Adaptive Thresholding calculates local "
            "thresholds for different image regions and can handle uneven lighting."
        ),
        (
            "Q4. What is the Watershed Algorithm? Why is it useful for overlapping objects?",
            "Watershed is a region-based segmentation algorithm that treats image "
            "intensities like a topographic surface. With suitable markers, it can "
            "identify boundaries between touching objects that might otherwise "
            "appear connected."
        ),
        (
            "Q5. How is K-Means Clustering applied to image segmentation?",
            "K-Means groups image pixels into K clusters based on feature similarity, "
            "often their color values. Each pixel is assigned to a cluster, and the "
            "cluster centers are used to create a segmented image."
        ),
        (
            "Q6. Compare threshold-based and clustering-based segmentation.",
            "Threshold-based methods separate pixels according to intensity values "
            "and threshold rules. Clustering-based methods group pixels by similarity "
            "in features such as color. Thresholding is often simpler, while clustering "
            "can represent multiple color groups."
        ),
        (
            "Q7. What challenges occur with complex backgrounds or varying illumination?",
            "Challenges include low contrast, shadows, noise, changing brightness, "
            "similar foreground and background colors, and irregular object boundaries. "
            "These factors can cause incorrect or incomplete segmentation."
        ),
        (
            "Q8. Mention five real-world applications of image segmentation.",
            "1. Medical image analysis. 2. Autonomous driving and road-scene analysis. "
            "3. Satellite image interpretation. 4. Industrial defect inspection. "
            "5. Object localization and image editing."
        ),
        (
            "Q9. How does segmentation improve object detection and image recognition?",
            "Segmentation isolates objects or regions of interest from the background. "
            "This helps systems analyze object boundaries, shapes and relevant pixels, "
            "which can improve localization and recognition."
        ),
        (
            "Q10. Compare traditional methods with U-Net and Mask R-CNN.",
            "Traditional methods include thresholding, Watershed and K-Means, which "
            "use manually selected rules or clustering. U-Net is a deep learning "
            "architecture commonly used for pixel-level semantic segmentation. "
            "Mask R-CNN performs instance segmentation by predicting a separate mask "
            "for each detected object. Deep learning methods generally need training "
            "data and more computational resources."
        )
    ]
```

```
for question, answer in qa:
    print("\n" + question)
    print("Answer:", answer)
    print("-" * 90)
```

```
Q1. What is image segmentation? Why is it fundamental in computer vision?
Answer: Image segmentation is the process of dividing an image into meaningful regions or obje
------------------------------------------------------------------------------------------

Q2. Differentiate between image segmentation and image classification.
Answer: Image classification assigns a class label to an image or its content. Image segmentat
------------------------------------------------------------------------------------------

Q3. Explain Global, Otsu's and Adaptive Thresholding.
Answer: Global Thresholding uses one fixed threshold for the whole image. Otsu's Thresholding
------------------------------------------------------------------------------------------

Q4. What is the Watershed Algorithm? Why is it useful for overlapping objects?
Answer: Watershed is a region-based segmentation algorithm that treats image intensities like
------------------------------------------------------------------------------------------

Q5. How is K-Means Clustering applied to image segmentation?
Answer: K-Means groups image pixels into K clusters based on feature similarity, often their c
------------------------------------------------------------------------------------------

Q6. Compare threshold-based and clustering-based segmentation.
Answer: Threshold-based methods separate pixels according to intensity values and threshold ru
------------------------------------------------------------------------------------------

Q7. What challenges occur with complex backgrounds or varying illumination?
Answer: Challenges include low contrast, shadows, noise, changing brightness, similar foregrou
------------------------------------------------------------------------------------------

Q8. Mention five real-world applications of image segmentation.
Answer: 1. Medical image analysis. 2. Autonomous driving and road-scene analysis. 3. Satellite
------------------------------------------------------------------------------------------

Q9. How does segmentation improve object detection and image recognition?
Answer: Segmentation isolates objects or regions of interest from the background. This helps s
------------------------------------------------------------------------------------------

Q10. Compare traditional methods with U-Net and Mask R-CNN.
Answer: Traditional methods include thresholding, Watershed and K-Means, which use manually se
------------------------------------------------------------------------------------------
```

```
print("EXPERIMENT 6: FINAL RESULT")
print("-" * 45)
print(
    "Image segmentation was implemented using Global Thresholding, "
    "Otsu's Thresholding, Adaptive Thresholding, Watershed Segmentation "
    "and K-Means Clustering."
)
print(
    "The output images were visualized and compared to study the "
    "characteristics and suitability of different segmentation techniques."
)
print(
    "The experiment demonstrates that the most suitable technique "
    "depends on image contrast, illumination, colors and object boundaries."
)
```

```
EXPERIMENT 6: FINAL RESULT
---------------------------------------------
Image segmentation was implemented using Global Thresholding, Otsu's Thresholding, Adaptive Th
The output images were visualized and compared to study the characteristics and suitability of
The experiment demonstrates that the most suitable technique depends on image contrast, illumi
```